

```
#pragma once

//Implement a circular queue of integers of user-specified size using a simple array.
//Provide routines to initialize(), Enqueue() and Dequeue() the queue.
//Make it thread safe.

namespace Griffis
{
    //The IQueue interface defines a contract for adding and removing
    //elements of parameterized type to a queue of runtime configurable size.
    template<class T = void*>
    class IQueue
    {
    public:
        //The IQueue interface exposes the following types
        typedef const int ErrorCodeType;
        typedef T EltType;

        //The IQueue interface exposes the following error codes
        static ErrorCodeType SUCCESS = 0;
        static ErrorCodeType FAIL = 1;
        static ErrorCodeType NOTINITIALIZED = 2;
        static ErrorCodeType ALREADYINITIALIZED = 3;
        static ErrorCodeType EMPTY = 4;
        static ErrorCodeType FULL = 5;

        //The Initialize method creates an empty queue of size queueSize.
        //that is, during normal operation the queue shall have a
        //maximum capacity of queueSize elements. It is the responsibility
        //of the interface user to guard against initializing the queue
        //with a negative number (which will be coerced into a very large
        //unsigned integer).
        //
        //Upon successful call to the initialize method, the queue shall
        //be ready for subsequent successful Enqueue/Dequeue operations.
        //Attempts to use an uninitialized queue shall not change the
        //state of the queue and return the NOTINITIALIZED error code.
        //Attempts to initialize an already initialized queue shall not
        //change the state of the queue and return the ALREADYINITIALIZED error code.
        //////////////////////////////////////
        virtual int Initialize(size_t queueSize) = 0;

        //The Enqueue method adds an element to the back of the queue.
        //The Enqueue method shall not modify the pass-by-reference parameter
        //If the queue is full, attempts to add more elements shall
        //not change the state of the queue and return the FULL error code
        //////////////////////////////////////
        virtual int Enqueue(EltType const& inVal) = 0;
    };
}
```

```
//The Dequeue method removes an element from the front of the queue.
//The Dequeue method shall conditionally modify the pass-by-reference
//parameter and assign it the value of that which has been removed from the queue
//If the queue is empty, attempts to remove an element shall not
//change the state of the queue, perform no operation on the
//out-param, and return the EMPTY error code.
////////////////////////////////////
virtual int Dequeue(ElType& outVal) = 0;

//Deleting an empty queue shall not result in memory leak.
//Deleting the non-empty queue provides no guarantee that all
//objects referred to by the queue will also be deleted.
////////////////////////////////////
virtual ~IQueue() = 0;

};
//in-line definition needed for pure virtual destructor.
template<class ElType> inline IQueue<ElType>::~~IQueue() {}

}
```

```
#pragma once

#include "IQueue.h"
#include "MyCriticalSection.h"

#ifndef OVERRIDE
#define OVERRIDE
#endif

namespace Griffis
{
    //forward declarations
    class ClippingCounter;
    template<class T> class RingPointer;

    //Exercise 2:
    //Implement a circular queue of integers of user-specified size using a simple array.
    //Provide routines to initialize(), enqueue() and dequeue() the queue.
    //Make it thread safe.
    //
    //The following class is a conditionally thread safe implementation of the IQueue interface
    //designed to meet the above criteria.
    //It utilizes only a "simple array" as its basis for storage.
    //The mechanism for thread safety serializes access to heap allocated elements that are themselves not
    //thread safe but are otherwise inaccessible except through the synchronized interface.
    //Class performs no exception handling, but is guaranteed to never get stuck in the critical section
    //Dynamic memory allocation used to accommodate (runtime) sizing of the queue.
    class CircularQueueOfInts
    {
        //complies to the contract prescribed by IQueue.h
        : public IQueue<int>
        {
            int* m_pStorage;           //simple array that stores the queue elements
            RingPointer<int>* m_pFrontOfQueue; //maintains the array index of the front of the queue
            RingPointer<int>* m_pBackOfQueue;  //maintains the array index of the back of the queue
            ClippingCounter* m_pNumInQueue;    //constrains the displacement of back of queue relative to front
            bool m_IsInitialized;             //flag indicating successful initialization
            MyCriticalSection m_CriticalSection; //CriticalSection used to serialize shared resource access

        public:
            CircularQueueOfInts();
            virtual ~CircularQueueOfInts() OVERRIDE;

            //The Initialize method heap allocates an empty queue of size queueSize.
            int Initialize(size_t queueSize) OVERRIDE;

            //The Enqueue method adds an element to the back of the queue.
            int Enqueue(int const& inVal) OVERRIDE;
```

```
        //The dequeue method removes an element from the front of the queue.
        int Dequeue(int& outVal) OVERRIDE;
    };
}
#undef OVERRIDE
```

```
//Implementation of CircularQueueOfInts

#include "RingPointer.h"
#include "ClippingCounter.h"
#include "CircularQueue.h"

#ifdef NULL
#define NULL 0
#endif

//no-fail constructor
Griffis::CircularQueueOfInts::CircularQueueOfInts()
: m_pStorage(NULL)
, m_pFrontOfQueue(NULL)
, m_pBackOfQueue(NULL)
, m_pNumInQueue(NULL)
, m_IsInitialized(false)
, m_CriticalSection()
{
    //no operation
}

//destructor
Griffis::CircularQueueOfInts::~CircularQueueOfInts()
{
    MyCriticalSectionLock locallyScopedLock(m_CriticalSection);
    delete[] m_pStorage;
    delete m_pFrontOfQueue;
    delete m_pBackOfQueue;
    delete m_pNumInQueue;
}

//Initialize method
//Accepts the runtime parameter that determines the queue size
//and initializes the object accordingly
//
//Returns a return code according to the IQueue contract.
//Uses a locally scoped CriticalSection to ensure serialized access.
//Method guaranteed to leave the critical section even in the case of an unhandled exception
int Griffis::CircularQueueOfInts::Initialize(size_t queueSize)
{
    //Lock the critical section
    MyCriticalSectionLock locallyScopedLock(m_CriticalSection);
    //Assume the object is already initialized
    int retVal = IQueue<int>::ALREADYINITIALIZED;
    if(!m_IsInitialized)
    {
        //otherwise, assume that the operation will fail
    }
}
```

```
//because the argument is invalid
retVal = IQueue<int>::FAIL;
if (queueSize > 0)
{
    //otherwise, heap allocate the objects that depend on runtime values
    //and then complete initialization successfully
    m_pStorage = new int[queueSize];
    m_pFrontOfQueue = new RingPointer<int>(m_pStorage, queueSize);
    m_pBackOfQueue = new RingPointer<int>(m_pStorage, queueSize);
    m_pNumInQueue = new ClippingCounter(queueSize);
    m_IsInitialized = true;
    retVal = IQueue<int>::SUCCESS;
}
}
return retVal;
//The critical section lock falls out of scope and exits the critical section
}

//Enqueue method
//When successful, the Enqueue method assigns the back of the queue
//to the value passed in and revises the queue structure accordingly
//
//Returns a return code according to the IQueue contract.
//Uses a locally scoped CriticalSection to ensure serialized access.
//Method guaranteed to leave the critical section even in the case of an unhandled exception.
int Griffis::CircularQueueOfInts::Enqueue(int const& inVal)
{
    //Lock the critical section
    MyCriticalSectionLock locallyScopedLock(m_CriticalSection);
    //Assume the object is not yet initialized
    int retVal = IQueue<int>::NOTINITIALIZED;
    if (m_IsInitialized)
    {
        //otherwise assume that the queue is full and will not accept new values
        retVal = IQueue<int>::FULL;
        if (!m_pNumInQueue->IsFull())
        {
            //otherwise assign the back of the queue to the value passed in
            m_pBackOfQueue->Dereference() = inVal;
            //ratchet the back of the queue one notch further away from the front of the queue
            m_pBackOfQueue->Increment();
            //keep track of the increase in size
            m_pNumInQueue->Increment();
            retVal = IQueue<int>::SUCCESS;
        }
    }
    return retVal;
    //The critical section lock falls out of scope and exits the critical section
}
```

```
}

//Dequeue method
//When successful, the Dequeue method assigns the the out value
//to that which is at the front of the queue and revises the
//queue structure accordingly
//
//Returns a return code according to the IQueue contract.
//Uses a locally scoped CriticalSection to ensure serialized access.
//Method guaranteed to leave the critical section even in the case of an unhandled exception.
int Griffiths::CircularQueueOfInts::Dequeue(int& outVal)
{
    //Lock the critical section
    MyCriticalSectionLock locallyScopedLock(m_CriticalSection);
    //Assume the object is not yet initialized
    int retVal = IQueue<int>::NOTINITIALIZED;
    if(m_IsInitialized)
    {
        //otherwise assume that the queue is empty
        retVal = IQueue<int>::EMPTY;
        if(!m_pNumInQueue->IsEmpty())
        {
            //otherwise assign the out value to what's at the front of the queue
            outVal = m_pFrontOfQueue->Dereference();
            //ratchet the front of the queue one notch closer to the back of the queue
            m_pFrontOfQueue->Increment();
            //keep track of the decrease in size
            m_pNumInQueue->Decrement();
            retVal = IQueue<int>::SUCCESS;
        }
    }
    return retVal;
    //The critical section Lock falls out of scope and exits the critical section
}

#undef NULL
```

```
#pragma once

//Violates the "do not use existing class libraries" rule
//Exception made on the premise that implementing a signalization algorithm
//manually is beyond the scope of the exercise.
#include <windows.h> //needed for critical section

//This pattern solves the problem of cleanly leaving the critical section
//even in cases of exceptions.
//
//pattern as shown in
//http://jrdodds.blogs.com/blog/2004/08/rail_in_c.html
namespace Griffis
{
    //The CriticalSection class wraps the Windows API calls
    //to provide a dispatch mechanism for the lock.
    class MyCriticalSection
    {
    public:
        MyCriticalSection()
        { ::InitializeCriticalSection(&m_Win32CritSec); }
        virtual ~CriticalSection()
        { ::DeleteCriticalSection(&m_Win32CritSec); }

        void Enter()
        { ::EnterCriticalSection(&m_Win32CritSec); }
        void Leave()
        { ::LeaveCriticalSection(&m_Win32CritSec); }

    private:
        MyCriticalSection(const MyCriticalSection&);
        MyCriticalSection& operator=(const MyCriticalSection&);
    };

    //CriticalSection lock enters the critical section on construction
    //and exits the critical section on destruction.
    //This enables serialize access to be bound to the lifespan of the
    //CriticalSection object, so even in cases of exceptions,
    //the CriticalSection as always exited when it falls out of scope.
    class MyCriticalSectionLock
    {
    public:
        MyCriticalSectionLock(MyCriticalSection& criticalSection)
```



```
    : m_Section(criticalSection)
    {
        m_Section.Enter();
    }

    virtual ~MyCriticalSection()
    {
        m_Section.Leave();
    }

private:
    MyCriticalSection(const MyCriticalSectionLock&);
    MyCriticalSectionLock& operator=(const MyCriticalSectionLock&);
};

}
```

```

#pragma once

#ifndef NULL
#define NULL 0
#endif

namespace Griffis
{
    //Ring Pointer
    //
    //Encapsulates wraparound behavior of a pointer to a simple array of
    //elements a parameterized type.
    //
    //Simple methods provide the ability to "ratchet" the pointer forward indiscriminately
    //and obtain the object pointed to.
    //For simplicity sake, the class does not do bounds checking.
    //
    //It is the responsibility of the class user to ensure
    //that the number of elements the ring pointer can point to is not greater than the size
    //of the array that the anchor element points to.
    //
    //It is the responsibility of the class user to ensure that the anchor element
    //passed in during construction time is the zeroeth [0] element of the array.
    template <class ElementType>
    class RingPointer
    {
        ElementType* m_pAnchorElement; //pointer to Zeroeth array element
        size_t m_Offset; //internal state determining position
        size_t m_NumElements; //size of the simple array

    public:
        typedef typename RingPointer::ElementType ElementType;
        RingPointer(ElementType* p, size_t numElements)
            : m_pAnchorElement(p)
            , m_Offset(0)
            , m_NumElements(numElements)
        {
            // no operation
        }

        //Dereference: return access to the current object
        //I elected not to overload operator* so as not to
        //appear as if offering a more robust facility than the method provided
        ElementType& Dereference() const
        {
            if (NULL == m_pAnchorElement)
            {
                //leaving nontrivial exception types out of scope for this exercise
            }
        }
    };
}

```

```
        throw 0;
    }
    ElementType* ptr(m_pAnchorElement + m_Offset);
    return *ptr;
}

//Increment: "ratchet" the ring pointer forward the next element in the array
//without concern for wraparound.
//I elected not to overload prefix operator++ so as not to
//appear as if offering a more robust facility than the method provided
RingPointer& Increment()
{
    if (NULL == m_pAnchorElement)
    {
        //leaving nontrivial exception types out of scope for this exercise
        throw 0;
    }
    //if the next array index is not the size of the array
    if(m_Offset < (m_NumElements-1))
    {
        ++m_Offset;
    }
    //otherwise, either out of bounds or incrementing will wrap around
    else
    {
        m_Offset = 0;
    }
    return *this;
}

private:
    // copy ops are private to prevent copying
    RingPointer(const RingPointer&);
    RingPointer& operator=(const RingPointer &);
    //hiding simple constructor provides rudimentary compile time
    //guard against bad usage
    RingPointer();
};

#ifdef NULL
```

```
#pragma once

namespace Griffis
{
    //Clipping counter keeps count and frees the user from
    //tracking exceedances of an upper or lower bound.
    //It maintains an internal value that incremented and
    //decremented but cannot be decremented below zero and
    //cannot be incremented above a configured threshold.
    class ClippingCounter
    {
        unsigned m_Count;
        unsigned m_Capacity;

    public:
        ClippingCounter(unsigned capacity)
            : m_Count(0)
            , m_Capacity(capacity)
        {
            //no operation
        }

        virtual ~ClippingCounter(){}

        bool IsFull() const
        {
            return m_Count == m_Capacity;
        }

        bool IsEmpty() const
        {
            return m_Count < 1;
        }

        //Increment method ratchets the counter one notch higher
        ClippingCounter& Increment()
        {
            if(m_Count < m_Capacity)
            {
                m_Count = m_Count + 1;
            }
            else
            {
                //explicitly set count to the maximum
                //even if it has exceeded the maximum
                m_Count = m_Capacity;
            }
            return *this;
        }
    };
}
```

```
}

//Decrement method ratchets the counter one notch lower
ClippingCounter& Decrement()
{
    if(m_Count >= m_Capacity)
    {
        m_Count = m_Capacity - 1;
    }
    else if (m_Count > 0)
    {
        --m_Count;
    }
    else
    {
        //explicitly set everything else zero
        //even in cases where the count is already zero
        m_Count = 0;
    }
    return *this;
}

private:
//hiding simple constructor provides rudimentary compile time
//guard against bad usage
ClippingCounter();

};

}
```

```
//This demo harness demonstrates the utility, operation, and thread safety of the
//CircularQueueOfInts class.

//Because it is a "demo harness," less consideration was put into making it
//"perfect." This means that I'm not worried about tchars and mixing
//wprintf and printf statements, lack of null checking, incomplete commenting,
//a random mixture of STL objects, etc.
//Also, the facilities used in the demo harness are considered
//outside the scope of the exercise and therefore are not subject to the
//"do not use existing libraries" clause.

////////////////////////////////////
// these classes are used to help set up a demo and exercise the solution
#include <windows.h>
#include <iostream>
#include <stdio.h>
#include <map>

////////////////////////////////////
//This is the solution to the exercise and the subject of the demo
#include "CircularQueue.h"
////////////////////////////////////

#ifdef NULL
#define NULL 0
#endif

using namespace Griffis;
using namespace std;

namespace Griffis
{
    struct MyConvenientStruct
    {
        IQueue<int>* iq;
        int threadNum;
        unsigned queueSize;
        bool polarity;
    };
}

void ThreadMain(MyConvenientStruct* s);
HANDLE CreateChildTh(MyConvenientStruct* s);

int main()
{
    unsigned myCount = 1;
```

```

const int numThr = 4;
cout << "Enter queue size: ";
cin >> myCount;
//CircularQueueOfInts will theoretically still work for large values,
//so adding a size limit within the class definition would be an arbitrary constraint.
//That being said, if the user inputs a negative value in the console
//it will be coerced into a very huge positive number.
//Here I message myCount to meet the usage agreement specified by the
//CircularQueueOfInts class description
//still, the user might also pick a really big value have to wait a long time,
//so for demo purposes I will help the user and constrain the value here
if (myCount > 31)
{
    myCount = 32;
}
wprintf(L"Using %d as queue size...\n", myCount);
IQueue<int>* iq = new CircularQueueOfInts();

//Four are threads given access to the same uninitialized pointer-to-IQueue
//the desire is to create a race on initializing the object as well as (ab)using it.
const bool DEQUEUE = false;
const bool ENQUEUE = true;
MyConvenientStruct s1 = {iq, 1, myCount, DEQUEUE};
MyConvenientStruct s2 = {iq, 2, myCount, ENQUEUE};
MyConvenientStruct s3 = {iq, 3, myCount, DEQUEUE};
MyConvenientStruct s4 = {iq, 4, myCount, ENQUEUE};

HANDLE hThreadHandle[numThr];
hThreadHandle[0] = CreateChildTh(&s1);
hThreadHandle[1] = CreateChildTh(&s2);
hThreadHandle[2] = CreateChildTh(&s3);
hThreadHandle[3] = CreateChildTh(&s4);

WaitForMultipleObjects(numThr, hThreadHandle, TRUE, INFINITE);

for(int i = 0; i < numThr; i++)
{
    if(CloseHandle(hThreadHandle[i]) != 0)
        wprintf(L"Closing the hThreadHandle[%d] handle is OK...\n", i);
    else
        wprintf(L"Failed to close the hThreadHandle[%d] handle, error %d...\n", GetLastError());
}

//delete via the interface, exercise virtual destructor.
delete iq;
system("pause");
return 0;
}

```

```
void ThreadMain(MyConvenientStruct* s)
{
    //set up a map for convenient error code reporting
    map<typename IQueue<int>::ErrorCodeType,string> m;
    m[IQueue<int>::SUCCESS] = "success";
    m[IQueue<int>::FAIL] = "fail";
    m[IQueue<int>::NOTINITIALIZED] = "not initialized";
    m[IQueue<int>::ALREADYINITIALIZED] = "already initialized";
    m[IQueue<int>::EMPTY] = "empty";
    m[IQueue<int>::FULL] = "full";

    //create a race on initializing the queue
    {
        string str = m[s->iq->Initialize(s->queueSize)];
        printf("thread %d init queue: %s \n", s->threadNum, &str[0]);
    }

    //create a race on the queue with equal numbers of competing threads
    for(unsigned x = 0; x < 2*s->queueSize; x++)
    {
        if (s->polarity)
        {
            string str = m[s->iq->Enqueue(x)];
            printf("thread %d enqueue-> %d: %s \n", s->threadNum, x, &str[0]);
        }
        else
        {
            int temp = -1;
            string str = m[s->iq->Dequeue(temp)];
            printf("thread %d dequeue-> %d: %s \n", s->threadNum, temp, &str[0]);
        }
    }

    HANDLE CreateChildTh(MyConvenientStruct* s)
    {
        HANDLE hThread;
        DWORD dwId;

        hThread = CreateThread(NULL,0,(LPTHREAD_START_ROUTINE)ThreadMain,(LPVOID)s,0,&dwId);

        if(hThread)
        {
            wprintf(L"CreateThread() is OK, ID num %d...\n", dwId);
            return hThread;
        }
        else
    }
```



```
{
    wprintf(L"CreateThread() failed, error %d\n", GetLastError());
    return NULL;
}

#ifdef NULL
```